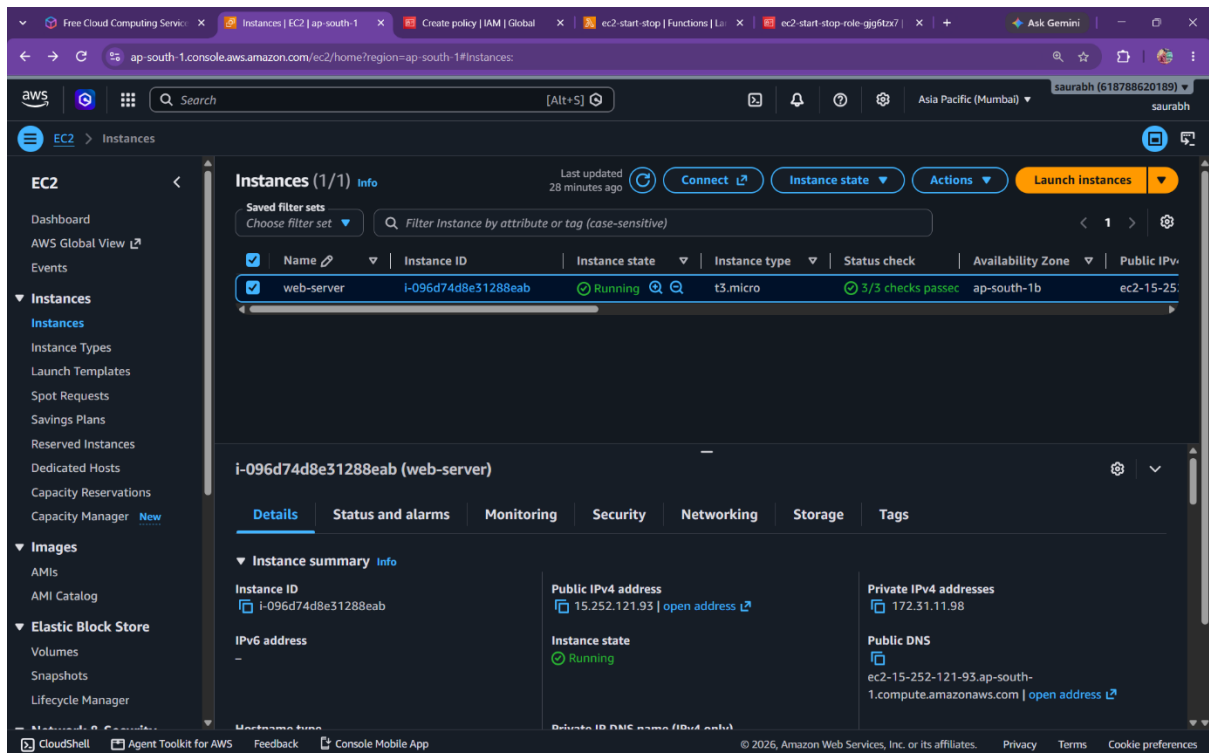


AWS EC2 Start/Stop Automation Using Lambda and EventBridge

Step 1: Create and Verify EC2 Instance

First, an **Amazon EC2 instance** was created and started successfully. The instance was checked from the EC2 console, where its status was shown as **Running**.

Screenshot: EC2 instance running.



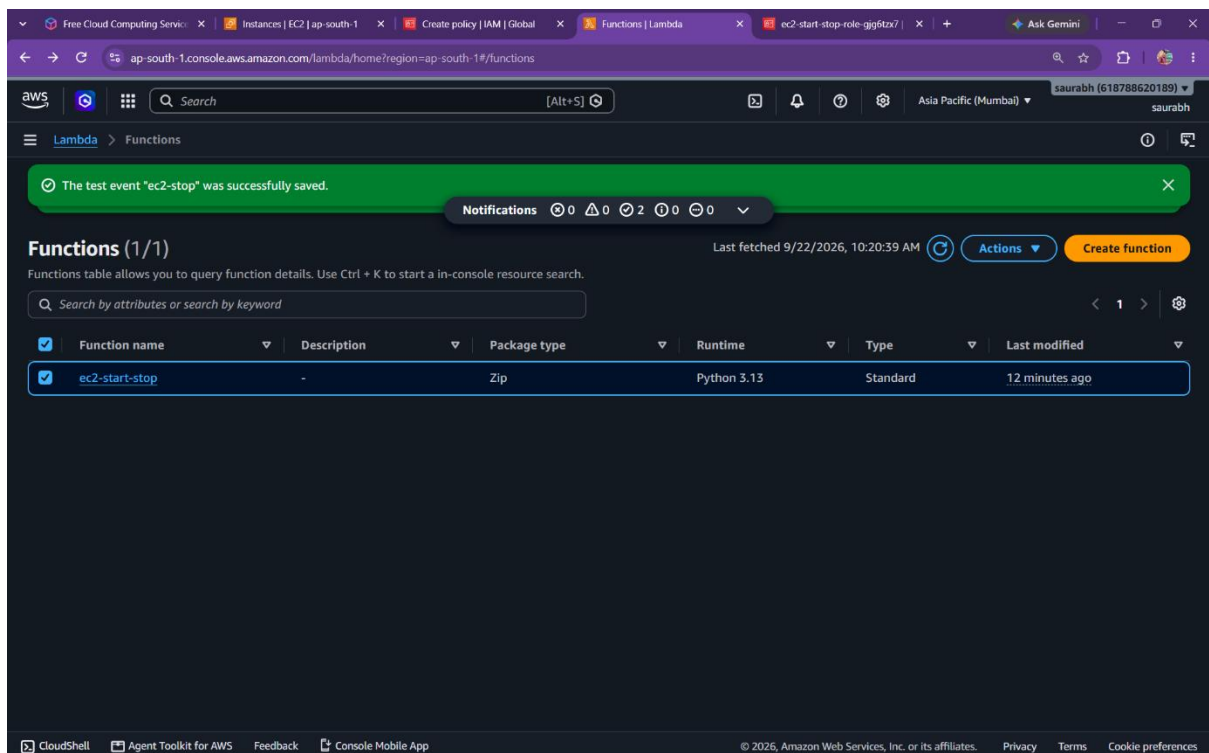
Step 2: Create AWS Lambda Function

An AWS Lambda function was created to automate the **start and stop operations of the EC2 instance**.

Two Lambda functions were configured:

- **EC2 Stop Function** – stops the EC2 instance.
- **EC2 Start Function** – starts the EC2 instance.

Screenshot: Lambda functions list.

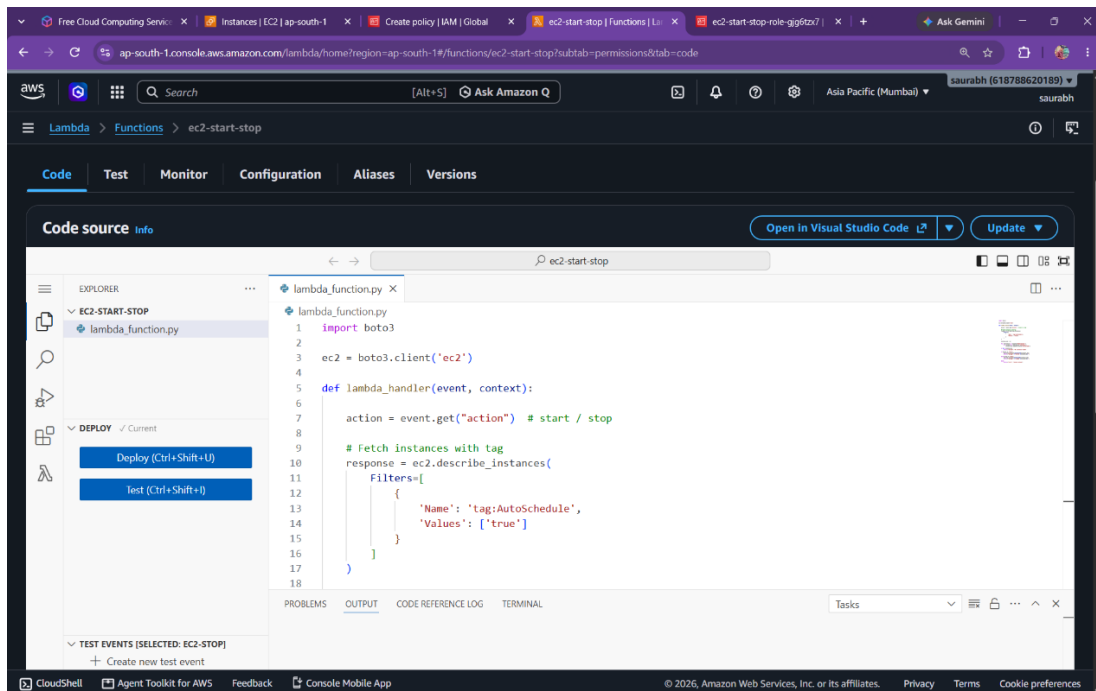
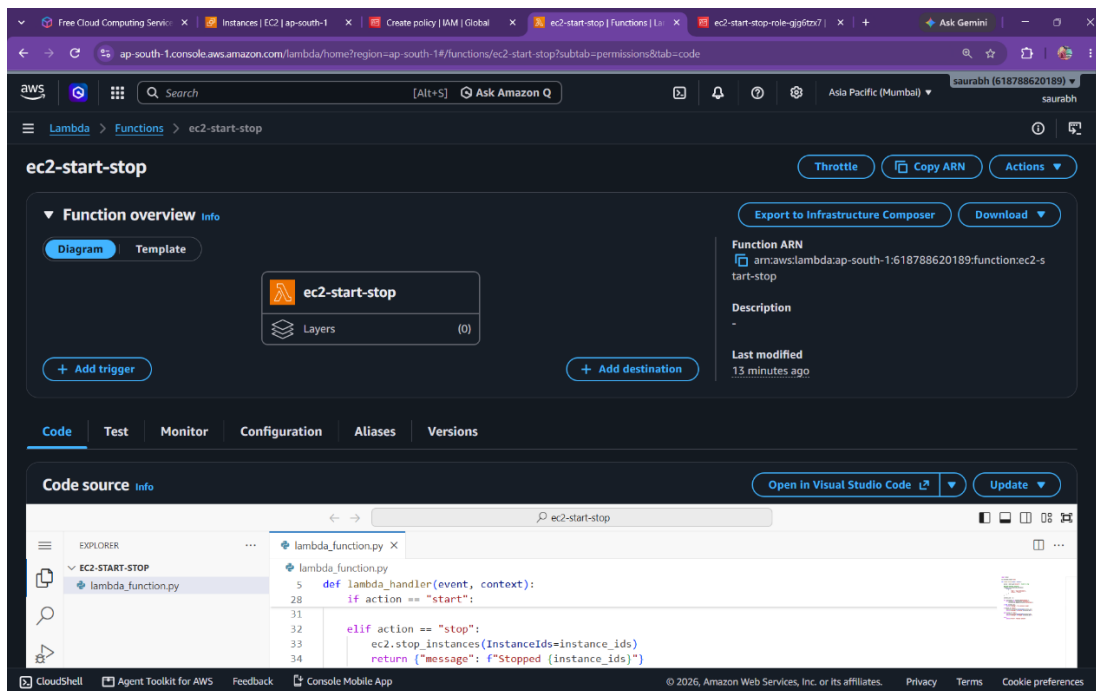


Step 3: Configure Lambda Function Code

The Lambda function code was configured to perform the required EC2 operation using the AWS SDK.

The code was written to identify the required EC2 instance and perform the **Stop Instances** or **Start Instances** operation.

Screenshot: Lambda function code.



Free Cloud Computing Service | Instances | EC2 | ap-south-1 | Create policy | IAM | Global | ec2-start-stop | Functions | Lambda | ec2-start-stop-role-gig6toz7 | Ask Gemini | saurabh (618788620189) | saurabh

ap-south-1.console.aws.amazon.com/lambda/home?region=ap-south-1#/functions/ec2-start-stop?subtab=permissions&tab=code

Search [Alt+S] Ask Amazon Q Asia Pacific (Mumbai)

Lambda > Functions > ec2-start-stop

+ Add trigger + Add destination Last modified 14 minutes ago

Code Test Monitor Configuration Aliases Versions

Code source info Open in Visual Studio Code Update

EXPLORER EC2-START-STOP lambda_function.py

DEPLOY Current Deploy (Ctrl+Shift+U) Test (Ctrl+Shift+I)

```
lambda_function.py
def lambda_handler(event, context):
    5
    20
    21     for reservation in response['Reservations']:
    22         for instance in reservation['Instances']:
    23             instance_ids.append(instance['InstanceId'])
    24
    25     if not instance_ids:
    26         return {"message": "No instances found"}
    27
    28     if action == "start":
    29         ec2.start_instances(InstanceIds=instance_ids)
    30         return {"message": f"Started {instance_ids}"}
    31
    32     elif action == "stop":
    33         ec2.stop_instances(InstanceIds=instance_ids)
    34         return {"message": f"Stopped {instance_ids}"}
    35
    36     else:
```

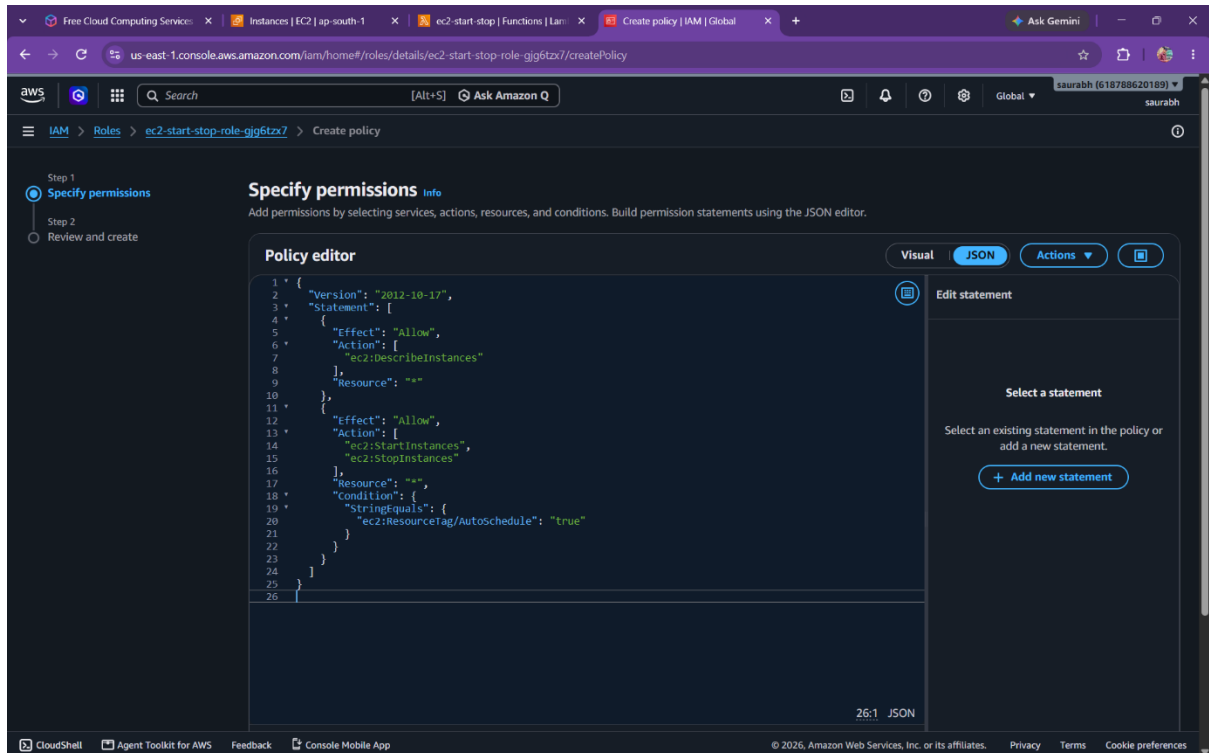
CloudShell Agent Toolkit for AWS Feedback Console Mobile App © 2026, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences

Step 4: Specify Lambda Permissions

The required permissions were configured for the Lambda function so that it could access and control the EC2 instance.

The Lambda function was given permission to perform EC2 start and stop operations.

Screenshot: Lambda permissions / execution role configuration.

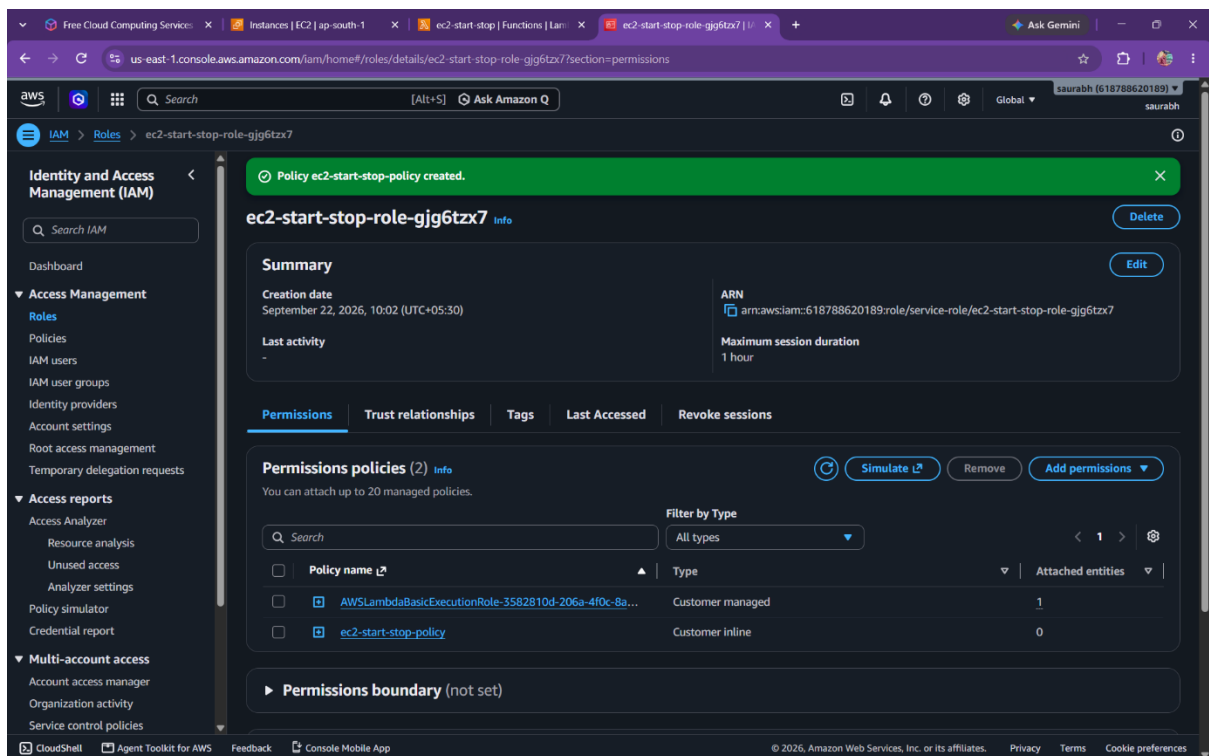


Step 5: Create EC2 Start/Stop IAM Role

An IAM role named **EC2 Start Stop Role** was created and attached to the Lambda function.

This role provides the required permissions for Lambda to perform actions on the EC2 instance.

Screenshot: EC2 Start/Stop IAM role and permissions.



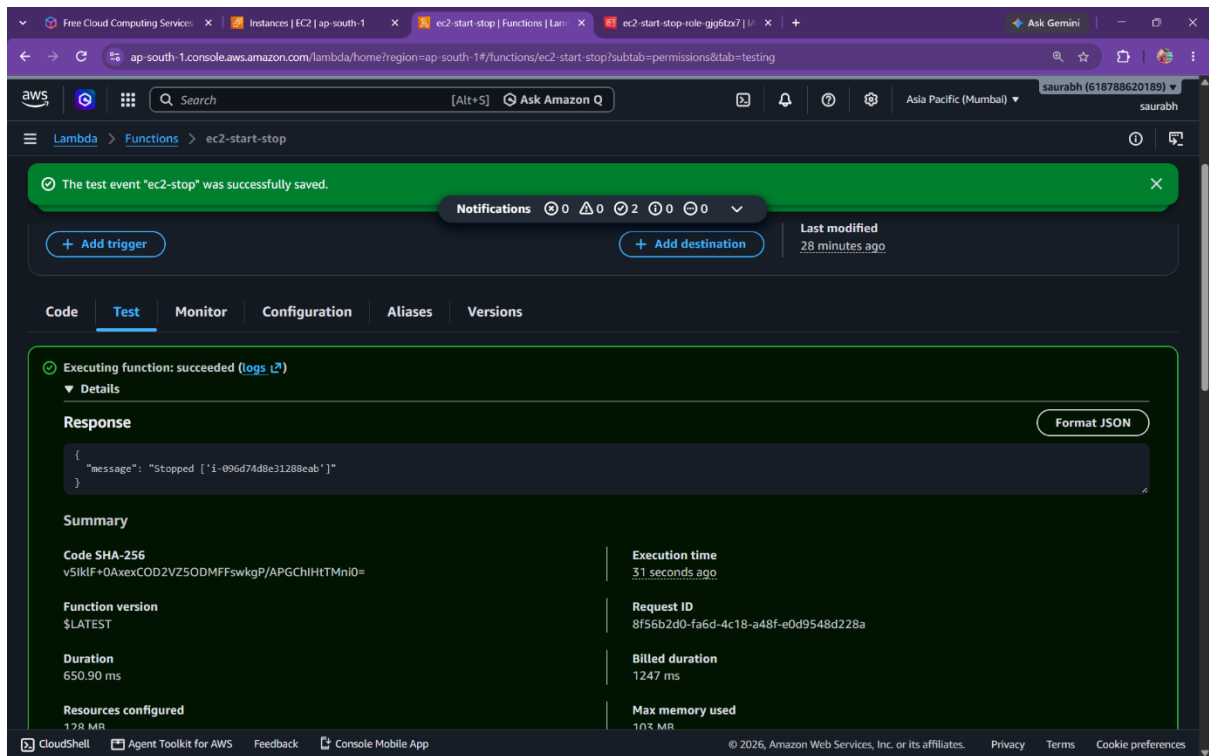
EC2 Stop Automation

Step 6: Execute EC2 Stop Lambda Function

The **EC2 Stop Lambda function** was invoked manually to test the automation.

The function successfully executed the EC2 stop operation.

Screenshot: Successful message showing that the EC2 instance was stopped.

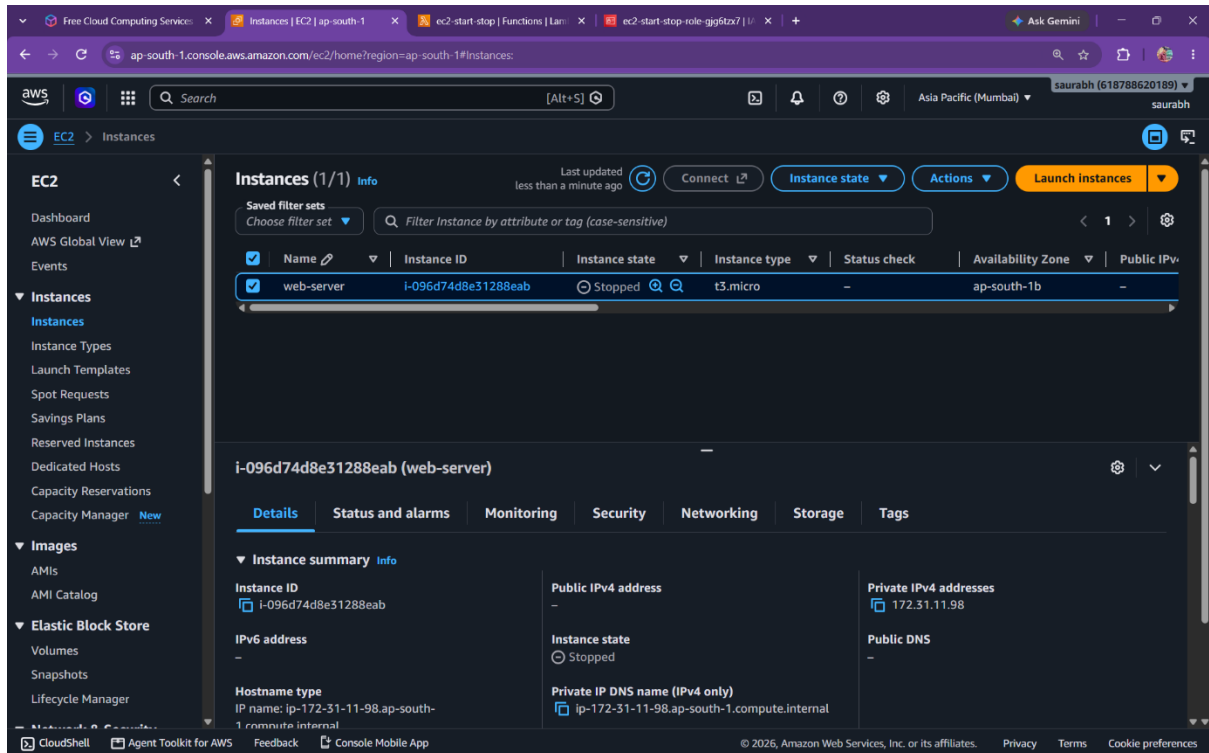


Step 7: Verify EC2 Instance Stopped

After executing the Lambda function, the EC2 console was checked.

The instance state changed from **Running** to **Stopped**, confirming that the Lambda function successfully stopped the EC2 instance.

Screenshot: EC2 instance stopped.



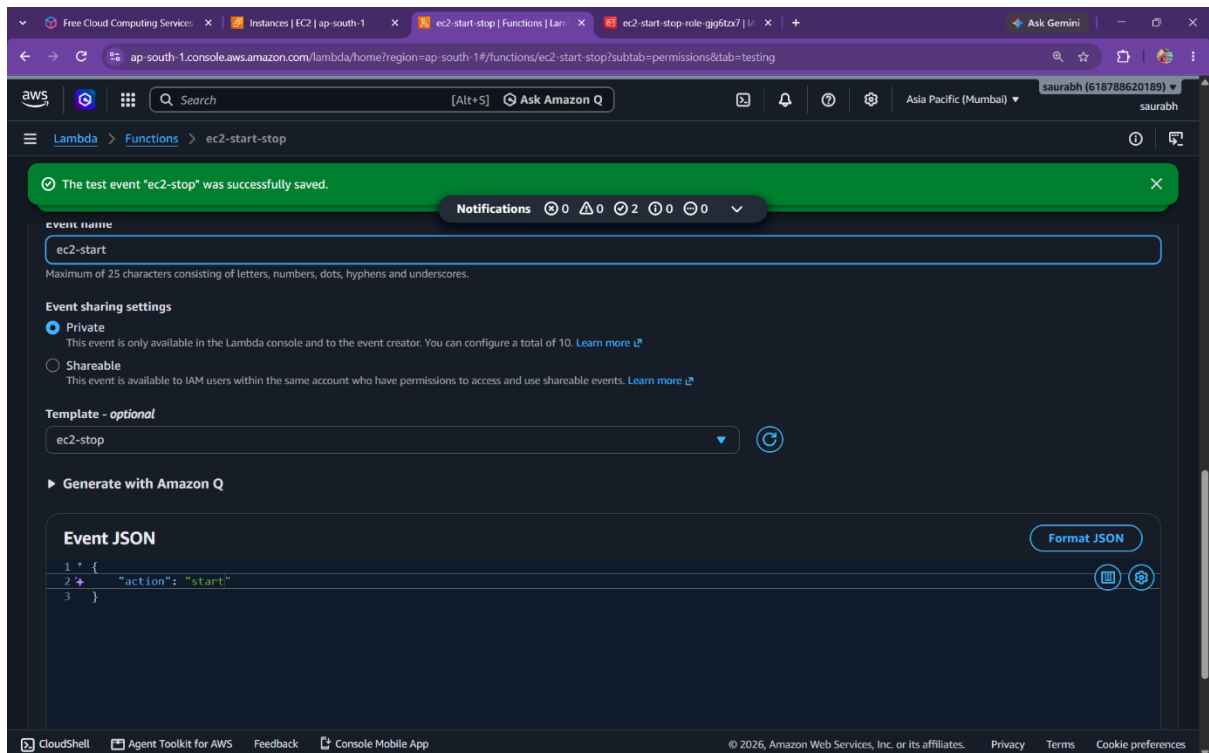
EC2 Start Automation

Step 8: Create/Configure EC2 Start Lambda Function

The **EC2 Start Lambda function** was configured to start the previously stopped EC2 instance.

The function uses the required EC2 permissions provided through the IAM role.

Screenshot: EC2 Start Lambda function.

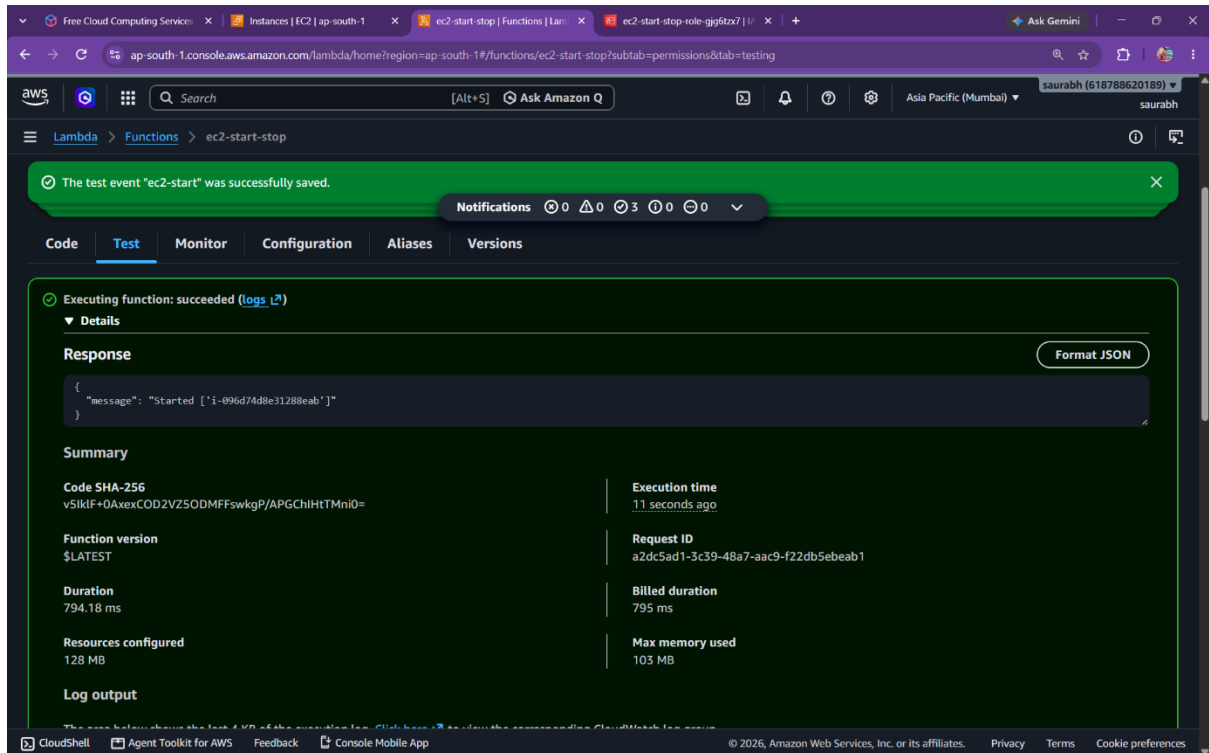


Step 9: Execute EC2 Start Lambda Function

The EC2 Start Lambda function was invoked to test the start operation.

The function executed successfully and generated a **successful instance-start message**.

Screenshot: Successful start message.

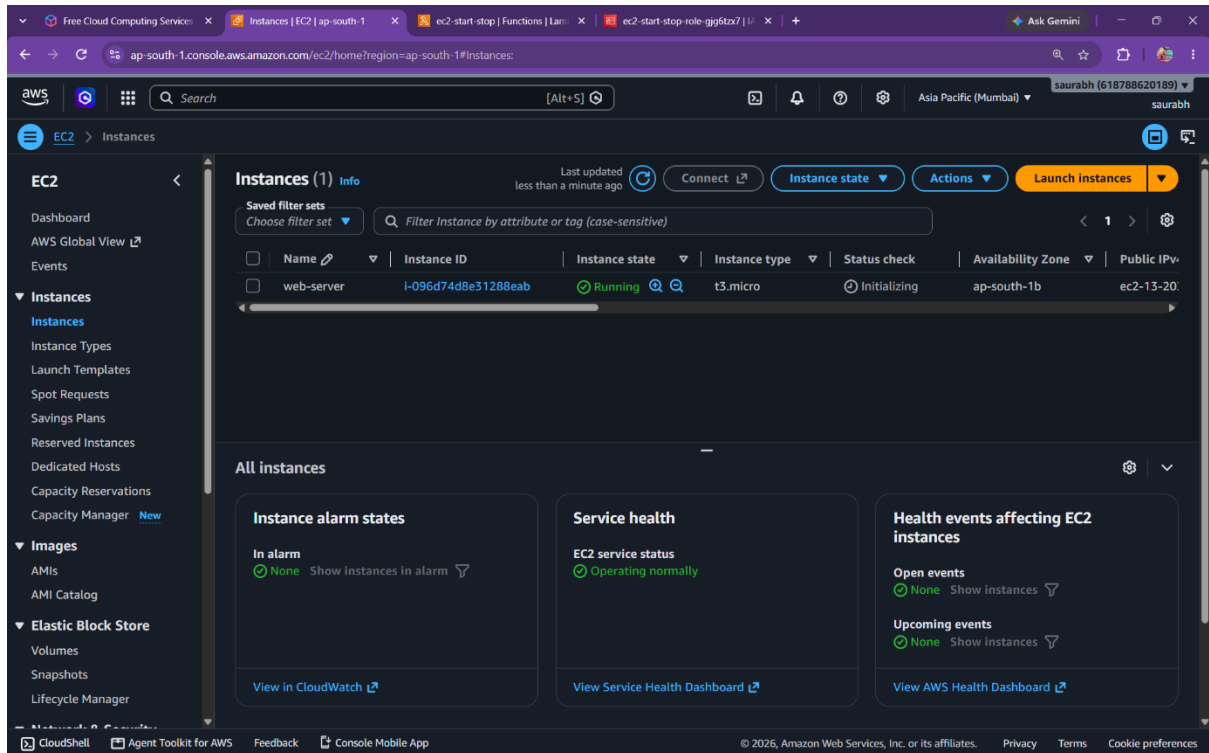


Step 10: Verify EC2 Instance Running

The EC2 console was opened to verify the result.

The instance state changed to **Running**, confirming that the Lambda function successfully started the EC2 instance.

Screenshot: EC2 instance running.



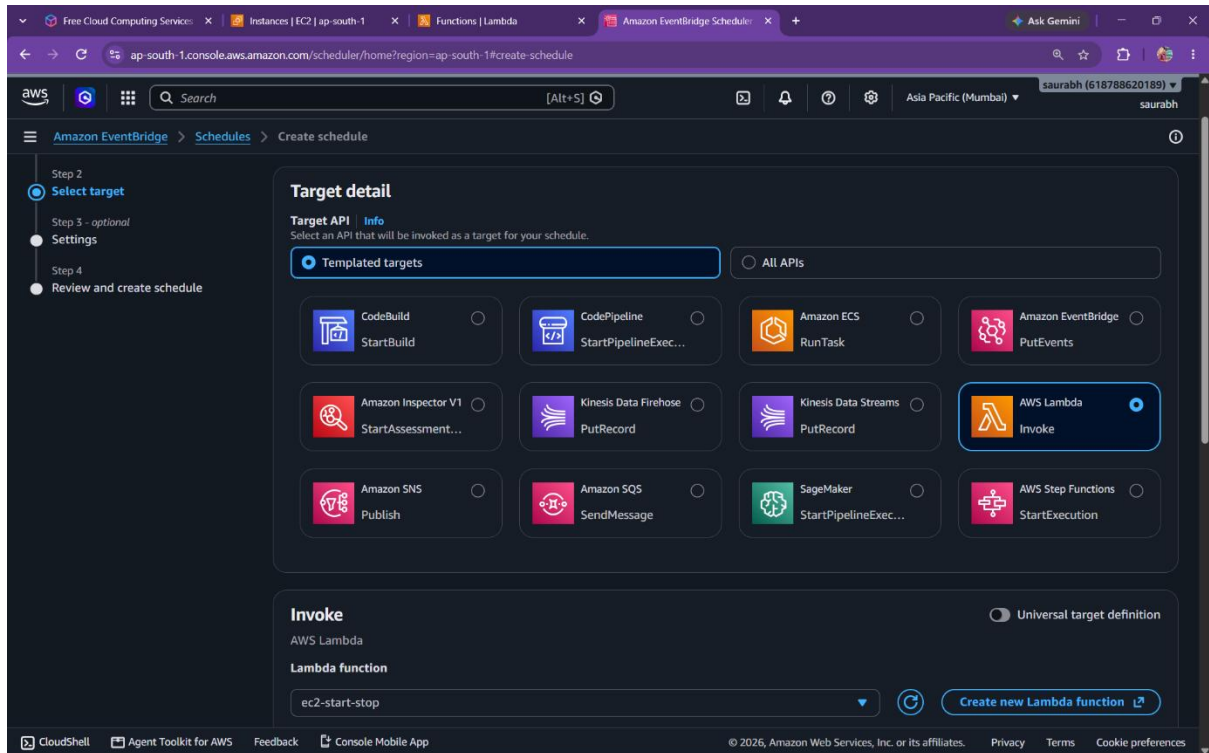
Event Bridge Scheduled Automation

Step 11: Create Amazon Event Bridge Rule

Amazon Event Bridge was configured to automate the Lambda execution according to a schedule.

A new Event Bridge rule was created with a **scheduled rule** configuration.

Screenshot: Event Bridge rule creation / target configuration.

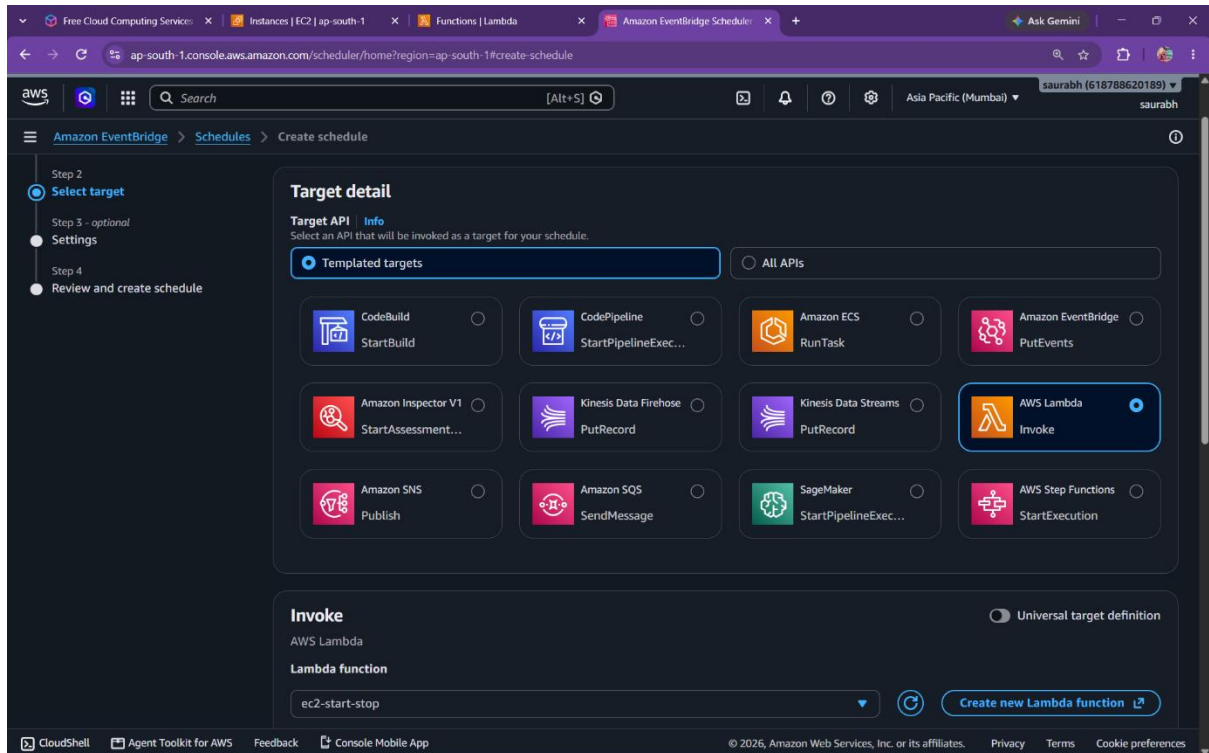


Step 12: Select Lambda as the Target

While configuring the Event Bridge rule, **Lambda Function** was selected as the target.

The required Lambda function was selected so that Event Bridge could automatically invoke it according to the configured schedule.

Screenshot: Event Bridge target selected as Lambda Invoke.



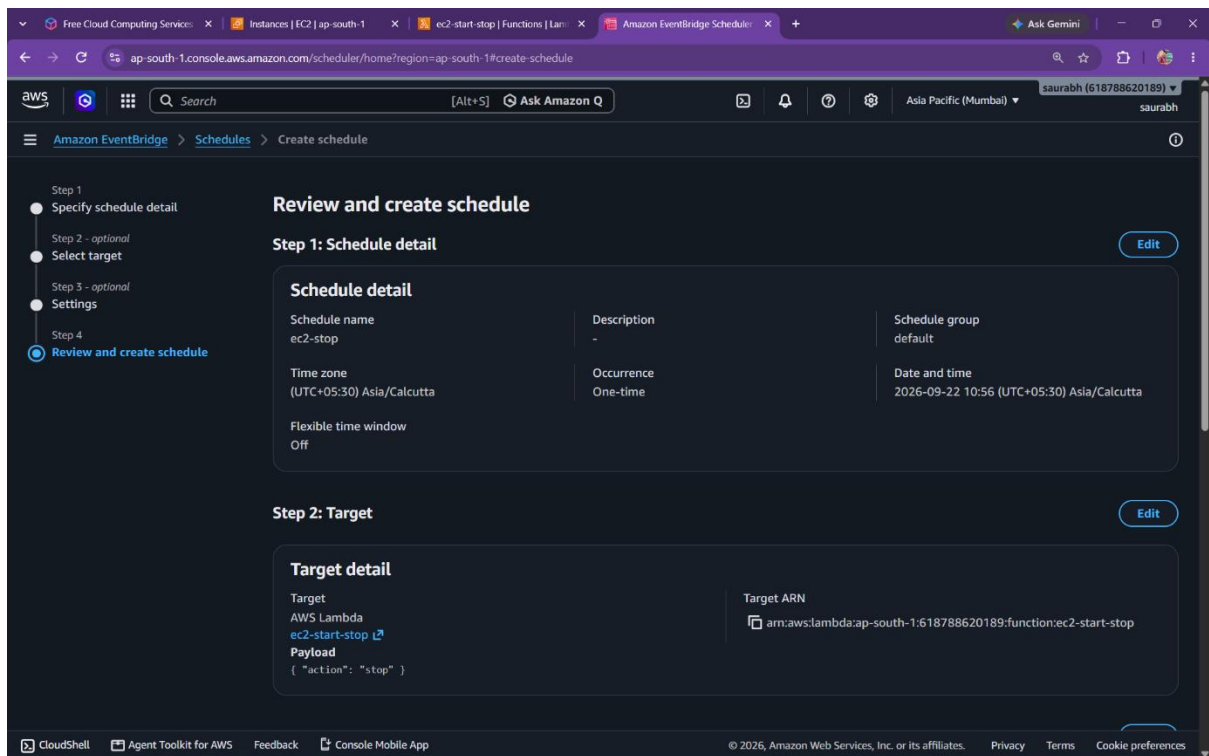
Step 13: Review and Create Schedule

The Event Bridge rule configuration was reviewed, including:

- Rule name
- Schedule
- Target Lambda function
- Invocation configuration

After verifying the settings, the schedule was created successfully.

Screenshot: Review and schedule configuration.



Step 14: Configure EC2 Stop Schedule

An Event Bridge schedule was configured to automatically invoke the **EC2 Stop Lambda function** at the specified time.

This allows the EC2 instance to be stopped automatically without manually executing the Lambda function.

Screenshot: EC2 stop schedule.

The screenshot displays the AWS Management Console interface for configuring an Amazon EventBridge schedule. The browser tabs at the top include 'Free Cloud Computing Services', 'Instances | EC2 | ap-south-1', 'ec2-start-stop | Functions | Lambda', and 'Amazon EventBridge Schedule'. The console URL is 'ap-south-1.console.aws.amazon.com/scheduler/home?region=ap-south-1#schedules/default/ec2-stop'.

The left-hand navigation pane shows the 'Amazon EventBridge' menu with options like Dashboard, Developer resources, Buses, Pipes, Scheduler, and Integration. The 'Scheduler' section is expanded, showing 'Schedules' as the active option.

The main content area shows the configuration for the 'ec2-stop' schedule. A green notification bar at the top states 'Your schedule ec2-stop is being created.' Below this, the 'ec2-stop' schedule is displayed with buttons for 'Disable', 'Edit', and 'Delete'.

The 'Schedule detail' section provides the following information:

Schedule name	Status	Schedule start time	Flexible time window
ec2-stop	Enabled	-	-

Description	Schedule ARN	Schedule end time	Created date
-	arn:aws:scheduler:ap-south-1:618788620189:schedule/default/ec2-stop	-	Sep 22, 2026, 10:55:38 (UTC+05:30)

Schedule group name	Execution time zone	Last modified date
default	Asia/Calcutta	Sep 22, 2026, 10:55:38 (UTC+05:30)

The 'Action after completion' is set to 'NONE'.

Below the details, there are tabs for 'Schedule', 'Target', 'Retry policy', 'Dead-letter queue', and 'Encryption'. The 'Schedule' tab is active, showing a 'One time schedule' set for 'Tue, Sep 22, 2026, 10:56:00 (UTC+05:30)'.

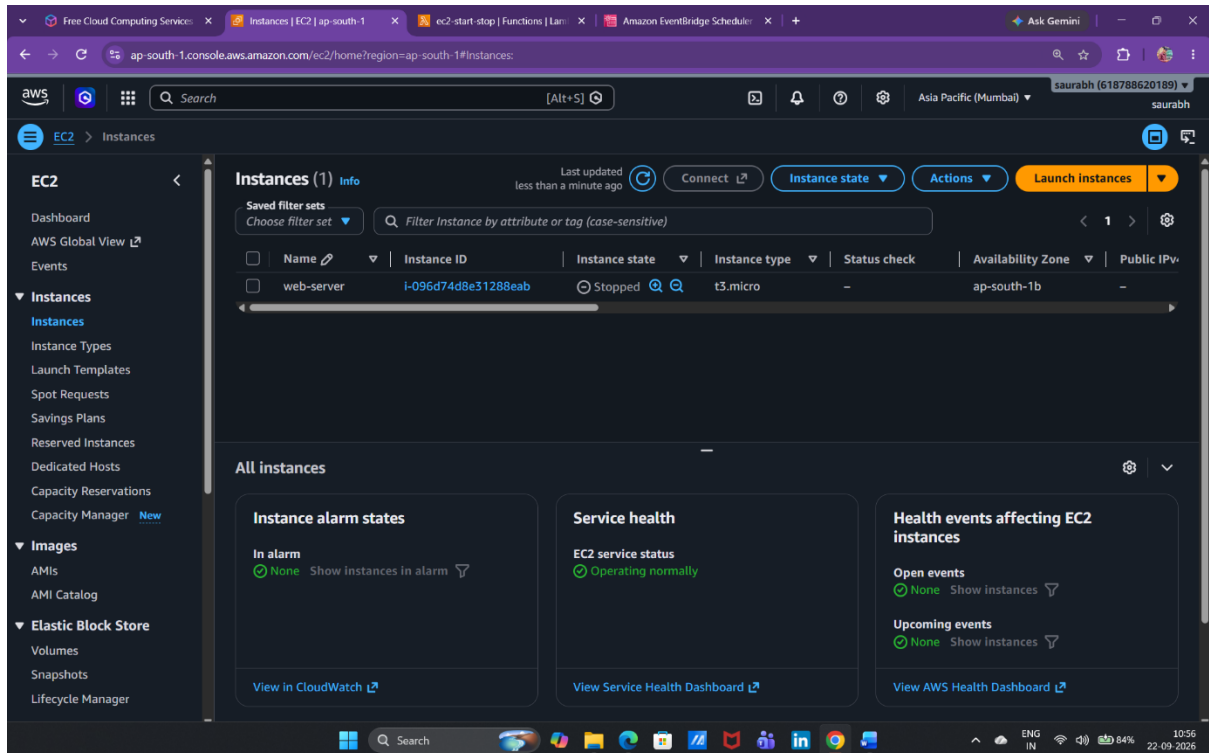
The footer of the console includes links for 'CloudShell', 'Agent Toolkit for AWS', 'Feedback', and 'Console Mobile App', along with copyright information for Amazon Web Services, Inc. or its affiliates.

Step 15: Verify Scheduled EC2 Stop

After the scheduled execution, the EC2 console was checked.

The instance was successfully changed to the **Stopped** state, confirming that Event Bridge successfully triggered the Lambda function.

Screenshot: EC2 instance stopped after scheduled execution.



CloudTrail Verification

Step 16: Verify Activity Using AWS CloudTrail

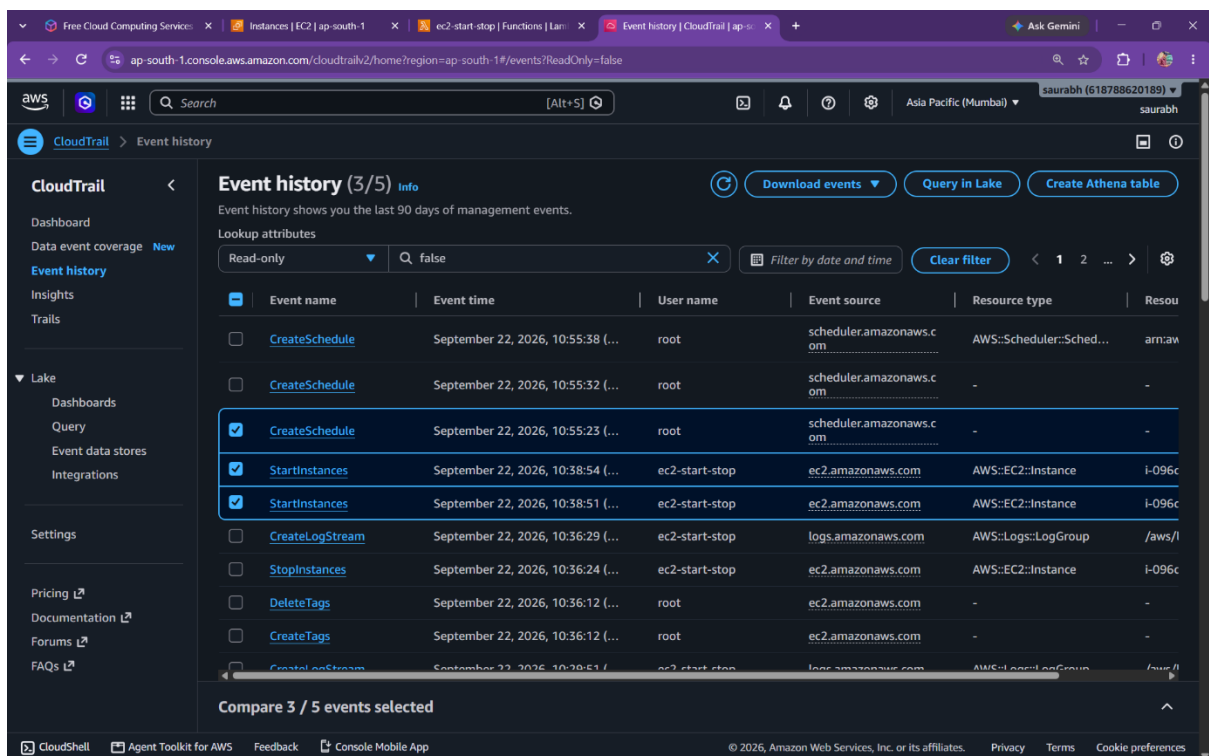
Finally, **AWS CloudTrail** was used to verify the activities performed during the project.

The **Event History** section was opened to view the recorded AWS API events related to the EC2 and Lambda operations.

The event history provides information such as:

- Event name
- Event time
- User/role
- AWS service
- Resource/action details

Screenshot: CloudTrail → Event History.



Final Project Flow➡

EC2 Instance Created & Running

↓

Lambda Stop Function Created

↓

IAM Permissions & EC2 Start/Stop Role Configured

↓

Lambda Stop Function Executed

↓

EC2 Instance Stopped

↓

Lambda Start Function Executed

↓

EC2 Instance Started

↓

Event Bridge Schedule Created

↓

Event Bridge Invokes Lambda Automatically

↓

EC2 Instance Stopped Automatically

↓

CloudTrail Event History Used for Verification